

Settings

[Configuration system](#) is a good way to configure the application on startup. In addition to the configurations, ABP provides another way to set and get some application settings.

A setting is a name-value pair stored in a dynamic data source, generally in a database. Setting system is extensible and there are pre-built providers for a user, a tenant, global and default.

Defining Settings

A setting must be defined before its use. ABP was designed to be [modular](#), so different modules can have different settings. A module must create a class derived from the [SettingDefinitionProvider](#) in order to define its settings. An example setting definition provider is shown below:

```
public class EmailSettingProvider : SettingDefinitionProvider
{
    public override void Define(ISettingDefinitionContext context)
    {
        context.Add(
            new SettingDefinition("Smtp.Host", "127.0.0.1"),
            new SettingDefinition("Smtp.Port", "25"),
            new SettingDefinition("Smtp.UserName"),
            new SettingDefinition("Smtp.Password", isEncrypted: true),
            new SettingDefinition("Smtp.EnableSsl", "false")
        );
    }
}
```

If you're developing a DDD module, you usually create this class in the [Domain](#) layer, of course, that's not mandatory.

ABP automatically discovers this class and registers the setting definitions.

SettingDefinition

[SettingDefinition](#) class has the following properties:

Name: Unique name of the setting in the application. This is **the only mandatory property**. Used to get/set the value of this setting in the application code (It's a good idea to define a const string for a setting name instead of using a magic string).

DefaultValue: A setting may have a default value.

DisplayName: A localizable string that can be used to show the setting name on the UI.

Description: A localizable string that can be used to show the setting description on the UI.

IsVisibleToClients: A boolean value indicates that whether this setting value is available in the client side or not. Default value is false to prevent accidentally publishing an internal critical setting value.

IsInherited: A boolean value indicates that whether this setting value is inherited from other providers or not. Default value is true and fallbacks to the next provider if the setting value was not set for the requested provider (see the setting value providers section for more).

IsEncrypted: A boolean value indicates that whether this setting value should be encrypted on save and decrypted on read. It makes possible to secure the setting value in the database.

Providers: Can be used to restrict providers available for a particular setting (see the setting value providers section for more).

Properties: A name/value collection to set custom properties about this setting those can be used later in the application code.

Change Setting Definitions of a Depended Module

In some cases, you may want to change some properties of a settings defined in some other module that your application/module depends on. A setting definition provider can query and update setting definitions.

The following example gets a setting defined by the [Volo.Abp.Emailing](#) package and changes its properties:

```
public class MySettingDefinitionProvider : SettingDefinitionProvider
{
    public override void Define(ISettingDefinitionContext context)
    {
        var smtpHost = context.GetOrNull("Abp.Mailing.Smtp.Host");
        if (smtpHost != null)
        {
            smtpHost.DefaultValue = "mail.mydomain.com";
            smtpHost.DisplayName =
                new LocalizableString(
                    typeof(MyLocalizationResource),
                    "SmtpServer_DisplayName"
                );
        }
    }
}
```

Using constants for the setting names is a good practice and ABP packages do it.

`Abp.Mailing.Smtp.Host` setting name is a constant defined by the `EmailSettingNames` class (in the `Volo.Abp.Emailing` namespace).

Reading the Setting Values

ISettingProvider

ISettingProvider is used to get the value of a setting or get the values of all the settings. Example usages:

```
public class MyService
{
    private readonly ISettingProvider _settingProvider;

    //Inject ISettingProvider in the constructor
    public MyService(ISettingProvider settingProvider)
    {
        _settingProvider = settingProvider;
    }

    public async Task FooAsync()
    {
        //Get a value as string.
        string userName = await _settingProvider.GetOrNullAsync("Smtp.UserName");

        //Get a bool value and fallback to the default value (false) if not set.
        bool enableSsl = await _settingProvider.GetAsync<bool>("Smtp.EnableSsl");

        //Get a bool value and fallback to the provided default value (true) if not set.
        bool enableSsl = await _settingProvider.GetAsync<bool>(
            "Smtp.EnableSsl", defaultValue: true);

        //Get a bool value with the IsTrueAsync shortcut extension method
        bool enableSsl = await _settingProvider.IsTrueAsync("Smtp.EnableSsl");

        //Get an int value or the default value (0) if not set
        int port = (await _settingProvider.GetAsync<int>("Smtp.Port"));

        //Get an int value or null if not provided
        int? port = (await _settingProvider.GetOrNullAsync("Smtp.Port"))?.To<int>();
    }
}
```

ISettingProvider is a very common service and some base classes (like **IApplicationService**) already property-inject it. You can directly use the **SettingProvider** property in such cases.

Reading Setting Values on the Client Side

If a setting is allowed to be visible on the client side, current value of the setting can also be read from the client code. See the following documents to understand how to get the setting values in different UI types;

[MVC / Razor Pages](#)

[Angular](#)

[Blazor](#)

Setting Value Providers

Setting system is extensible, you can extend it by defining setting value providers to get setting values from any source and based on any condition.

ISettingProvider uses the setting value providers to obtain a setting value. It fallbacks to the next value provider if a value provider can not get the setting value.

There are 5 pre-built setting value providers registered by the order below:

DefaultValueSettingValueProvider : Gets the value from the default value of the setting definition, if set (see the [SettingDefinition](#) section above).

ConfigurationSettingValueProvider : Gets the value from the [IConfiguration](#) service.

GlobalSettingValueProvider : Gets the global (system-wide) value for a setting, if set.

TenantSettingValueProvider : Gets the setting value for the current tenant, if set (see the [multi-tenancy](#) document).

UserSettingValueProvider : Gets the setting value for the current user, if set (see the [current user](#) document).

Setting fallback system works from bottom (user) to top (default).

Global, Tenant and User setting value providers use the **ISettingStore** to read the value from the data source (see the section below).

Setting Values in the Application Configuration

As mentioned in the previous section, **ConfigurationSettingValueProvider** reads the settings from the **IConfiguration** service, which can read values from the **appsettings.json** by default. So, the easiest way to configure setting values to define them in the **appsettings.json** file.

For example, you can configure **ISmtpSender** settings as shown below:

```
{
  "Settings": {
    "Abp.Mailing.DefaultFromAddress": "noreply@mydomain.com",
    "Abp.Mailing.DefaultFromDisplayName": "My Application",
    "Abp.Mailing.Smtp.Host": "mail.mydomain.com",
    "Abp.Mailing.Smtp.Port": "547",
    "Abp.Mailing.Smtp.UserName": "myusername",
    "Abp.Mailing.Smtp.Password": "mySecretPassW00rd",
    "Abp.Mailing.Smtp.EnableSsl": "True"
  }
}
```

Setting values should be configured under the **Settings** section as like in this example.

IConfiguration is an .NET Core service and it can read values not only from the **appsettings.json** , but also from the environment, user secrets... etc. See Microsoft's documentation for more.

Encrypting Setting Values in the Application Configuration

You have to use the **ISettingEncryptionService** to encrypt the setting values when the **IsEncrypted** property of a setting definition is set to **true** , which means that the setting value should be encrypted in the **appsettings.json** .

Example of injecting the **ISettingEncryptionService** into a class and encrypting the plain text:

```
public class MyService
{
    private readonly ISettingEncryptionService _encryptionService;

    public MyService(ISettingEncryptionService encryptionService)
    {
        _encryptionService = encryptionService;
    }

    public void EncryptValue(string plainText)
    {
        var ciphertext = _encryptionService.Encrypt(plainText);
    }
}
```

Then, Replace **ENCRYPTED_VALUE_HERE** with the ciphertext generated by **ISettingEncryptionService.Encrypt** method.

```
{
  "Settings": {
    "MySetting": "ENCRYPTED_VALUE_HERE"
  }
}
```

Custom Setting Value Providers

If you need to extend the setting system, you can define a class derived from the **SettingValueProvider** class. Example:

```
public class CustomSettingValueProvider : SettingValueProvider
{
    public override string Name => "Custom";
}
```

```

public CustomSettingValueProvider(ISettingStore settingStore)
    : base(settingStore)
{
}

public override Task<string> GetOrNullAsync(SettingDefinition setting)
{
    /* Return the setting value or null
       Use the SettingStore or another data source */
}
}

```

Alternatively, you can implement the `ISettingValueProvider` interface. Remember to register it to the dependency injection in this case.

Every provider should have a unique Name (which is "Custom" here). Built-in providers use the given names:

```

DefaultSettingValueProvider : "D".
ConfigurationSettingValueProvider : "C".
GlobalSettingValueProvider : "G".
TenantSettingValueProvider : "T".
UserSettingValueProvider : "U".

```

One-letter names were preferred to reduce the data size in the database (provider name is repeated in each row).

Once you define a custom setting value provider, you need to explicitly register it to the `AbpSettingOptions` :

```

Configure<AbpSettingOptions>(options =>
{
    options.ValueProviders.Add<CustomSettingValueProvider>();
});

```

This example adds it as the last item, so it will be the first value provider used by the `ISettingProvider` . You could add it to another index in the `options.ValueProviders` list.

ISettingStore

While a setting value provider is free to use any source to get the setting value, the `ISettingStore` service is the default source of the setting values. Global, Tenant and User setting value providers use it.

ISettingEncryptionService

`ISettingEncryptionService` is used to encrypt/decrypt setting values when `IsEncrypted` property of a setting definition was set to `true`.

You can replace this service in the dependency injection system to customize the encryption/decryption process. Default implementation uses the `StringEncryptionService` which is implemented with the AES algorithm by default (see string [encryption document](#) for more).

Setting Management Module

The core setting system is pretty independent and doesn't make any assumption about how you manage (change) the setting values. Even the default `ISettingStore` implementation is the `NullSettingStore` which returns null for all setting values.

The setting management module completes it (and implements `ISettingStore`) by managing setting values in a database. See the [Setting Management Module document](#) for more.